TNE20003 – INTERNET AND CYBERSECURITY FOR ENGINEERING APPLICATIONS

Portfolio Task – Project Aim for Credit

Name: Van Praseth Tuy

ID:104482107

Table of Contents

1. Introduction 2

2. MQTT Protocol Overview 3

3. Implementation 3

3.1 Assessment Pass 3

3.2 Assessment Credit 10

4.Access is blocked from off-site 13

5.Security Issues 13

5.Conclusion 14

6.Reference 15

1. Introduction

In this project aim to integrate knowledge for a engineering/science problem within an IoT environment to develop prototype solution, using the MQTT(Message Queuing Telemetry Transport) protocol as the solution to manage data transfer between device. In the digital world, the Internet of Things (IoT) has transformed the way devices communicate and interact with each other.

In my current implementation plan for this project, I want create topic publish the weather information to the subcriber, this could provide an update information to the client who subcribe to that topic.

The MQTT is the best protocol allow real time data exchange across various devices globally is crucial in today's interconnected world. This capability allows systems to provide up to date information, ensuring efficient operation and quick response times information, such as tempreture, healthcare, and transportation. One of the most important methods for transferring data to multiple device in such scenarios is the MQTT protocol. Moreover, this is a lightweight protocol is designed specifically for IoT environments, allowing devices to communicate efficiently with minimal bandwidth and resource usage, acting as a broker in exchnaging process, MQTT facilitates the efficient transfer of messages between devices.

2. MQTT Protocol Overview

MQTT (Message Queuing Telemetry Transport) is a simple messaging protocol. It is designed for use in IoT (Internet of Things) and machine to machine communication, especially in networks with low bandwidth. MQTT is lightweight and works well for remote devices, helping them communicate efficiently by using a publish/subscribe model over TCP/IP. The MQTT is a central point that connected the device with a spectific topic. The MQTT broker acts as the central hub that connects devices through specific topics. Moreover, it manages communication between clients device by using a publish/subscribe model. When a publisher device sends a message under a particular topic that has been define by the publisher, then the broker receives this message and delivers the data to other subscribers devices that have subscribed to that topic that defined topic. This process allows efficient communication between devices through the topics created by publishers.

There are three main components involved in the communication process:

- MQTT Client: The MQTT client is any device that connected to MQTT broker and could act as a publisher when sending messgae and subscriber when reciving message.
- MQTT Broker: The MQTT broker is the communication management betweet the device, it responsible for receiving and forwarding messages and authentication client.
- MQTT Connection: The MQTT connection is begin a client sends a CONNECT message to the broker. Both the client and broker communicate using TCP/IP. Clients always connect through the broker, not directly with each other, so the broker manages all messages between them.

3. Implementation

3.1 Assessment Pass

The implementation for the pass level involves generating random data on device 1 and posting messages to a private topic on the MQTT server, using the second device as the subscriber to that private topic. Both devices communicate with each other using

the MQTT server as a central hub. In addition, device 2 also subscribes to the public channel and prints all public messages along with their sub topic information. To implement to that part, I have to through these step:

- Implement a Publisher Client

Firstly, I have used the Paho MQTT library for python and set up the MQTT broker rule28.i4t.swin.edu.au, username, password (<104482107>)and port number<1883>. Then, I have define private topic of temprature where the client publishes.

```
broker = "rule28.i4t.swin.edu.au"
username = "<104482107>"
password = "<104482107>"
port = 1883
private_topic = f"{username}/temperature"
```

Addtionally, I created an MQTT client with a unique ID "Device1" and set up authentication using the username and password for secure access. Then, i connect to the MQTT broker with a port number and keep the connection 60 second alive.

```
client = mqtt.Client(client_id="Device1")
client.username_pw_set(username, password)
client.connect(broker, port, 60)
```

Lastly, I just created a loop to generate the tempreture data between 15°C-35 °C and pause every 5 seconds between each data update.

```
client.loop_start()

try:
    while True:
        temperature = random.uniform(15, 35)
        format_temperature = f"{temperature:.2f}"
        client.publish(private_topic, f"Temperature: {format_temperature} °C")
        print(f"Published: Forecast Temperature: {format_temperature} °C to topic: {private_topic}")
        time.sleep(5)
finally:
    client.loop_stop()
    client.disconnect()
```

- Implement a Subscriber Client:

Secondly, I have implemented a second device used as a subscriber to the topic. Intially, I just define the broker, username, password and port number the same as the

device 1 and the topic where this client subscribes for messages.

```
broker = "rule28.i4t.swin.edu.au"
username = "<104482107>"
password = "<104482107>"
port = 1883
public_topic = "public/#"
private_topic = f"{username}/temperature"
```

Then, I created an on_connect function to show the connection status. If the connection is successful, the client subscribes to its private topic and all public topics. If the connection fails, it shows an error message.

```
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("The Device 2 connected successfully")
        client.subscribe(public_topic)
        client.subscribe(private_topic)
    else:
        print(f"The Device 2 failed to connect, return code {rc}")
```

Moreover, I created an on_message function to execute when the client receives a message, displaying the message content along with the topic it was received from. And, i just created an MQTT client with a unique ID "Device2" with the authentication to verify the user name and password. Then, I set up callback functions for handling connections and messages. The client connects to the broker on port 1883 with a 60 second keep alive and runs using client.loop_forever().

```
def on_message(client, userdata, msg):
    print(f"The Device 2 received a message: {msg.topic} - {msg.payload.decode()}")

client = mqtt.Client(client_id="Device2")
client.username_pw_set(username, password)
client.on_connect = on_connect
client.on_message = on_message

client.connect(broker, port, 60)

client.loop_forever()
```

This demonstrates the communication process, where Device 1 acts as the publisher and Device 2 as the subscriber for both private and public topic, showcasing how they interact through the MQTT broker.

1. Run Device 1:

- Open a terminal and run the code for Device 1.

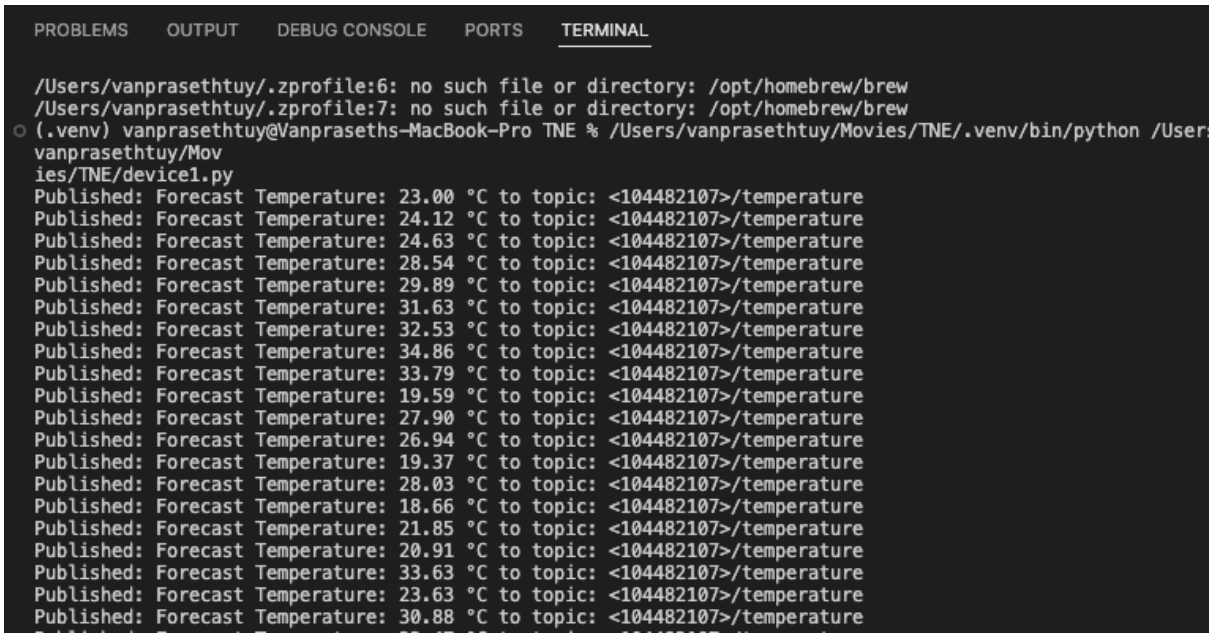
- Device 1 will generate temperature data and publish it to the private topic MQTT topic.

2. Run Device 2:

- Open another terminal and run the code for Device 2.
- Device 2 will then receive the messages being published by Device 1 and display them on the terminal.

  Noted: To run this code you have to used Swinburne network or using Swinburne VPN.

- MQTT client received message from device 1



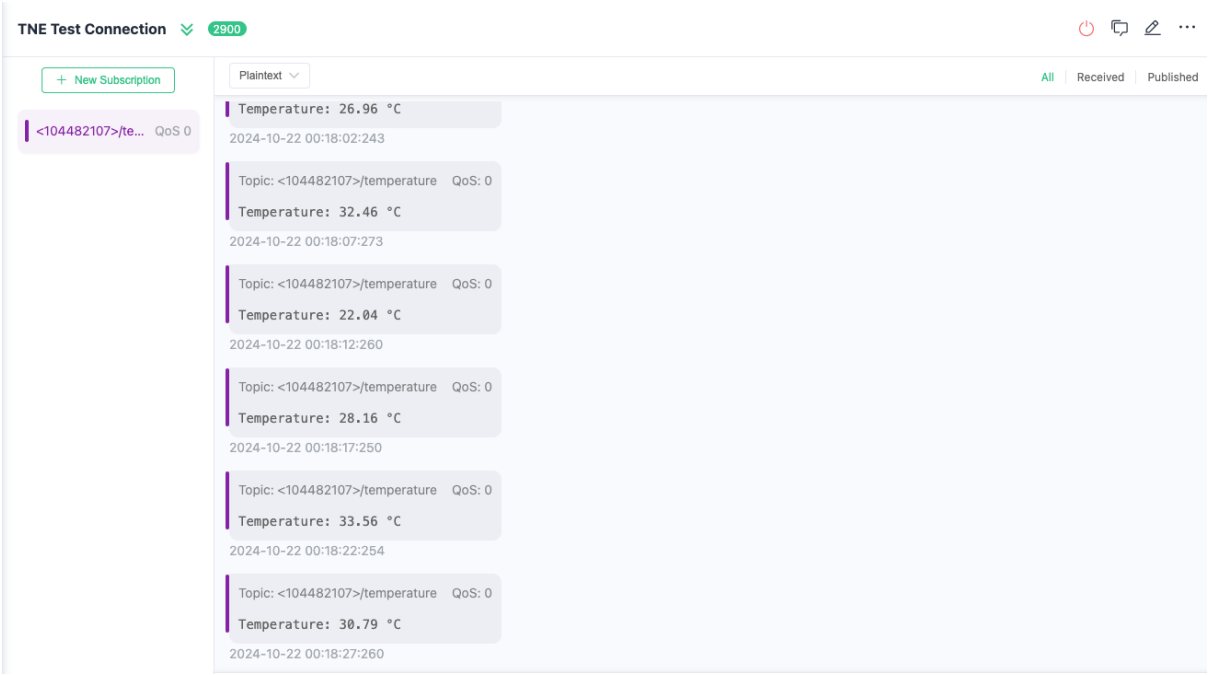
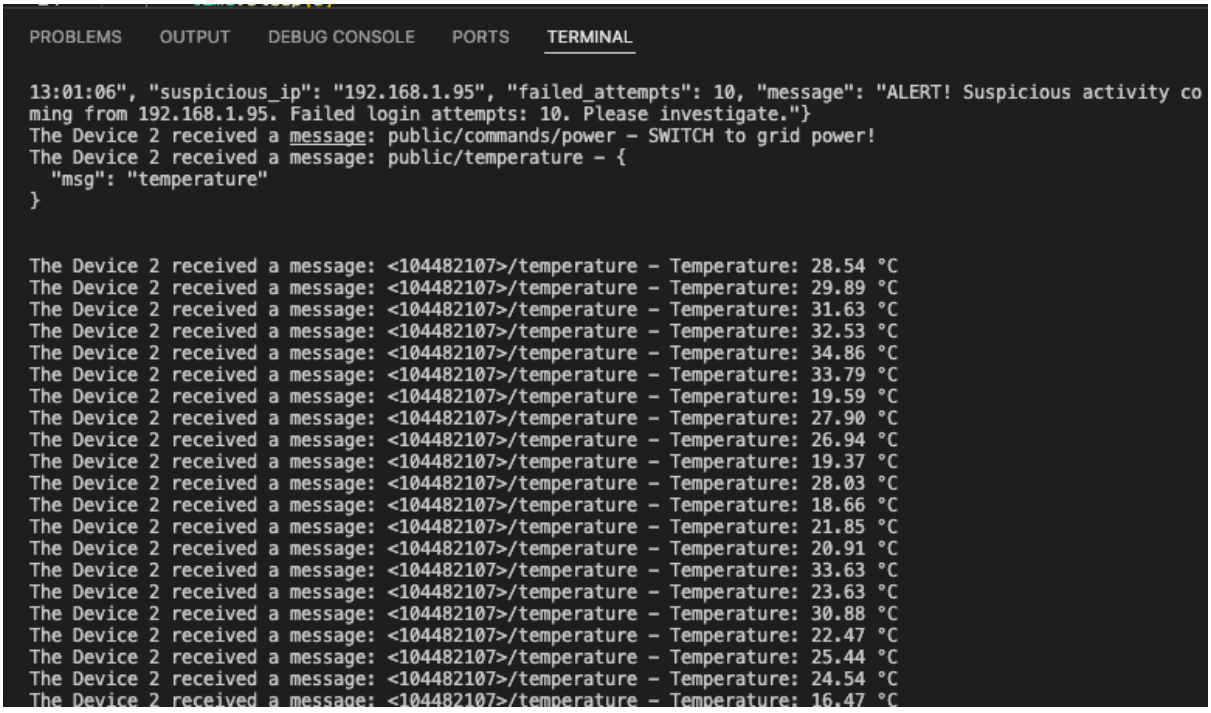


Figure 1 Device 2 receive message on (MQTT)



- MQTT client generating a message and send to device 2

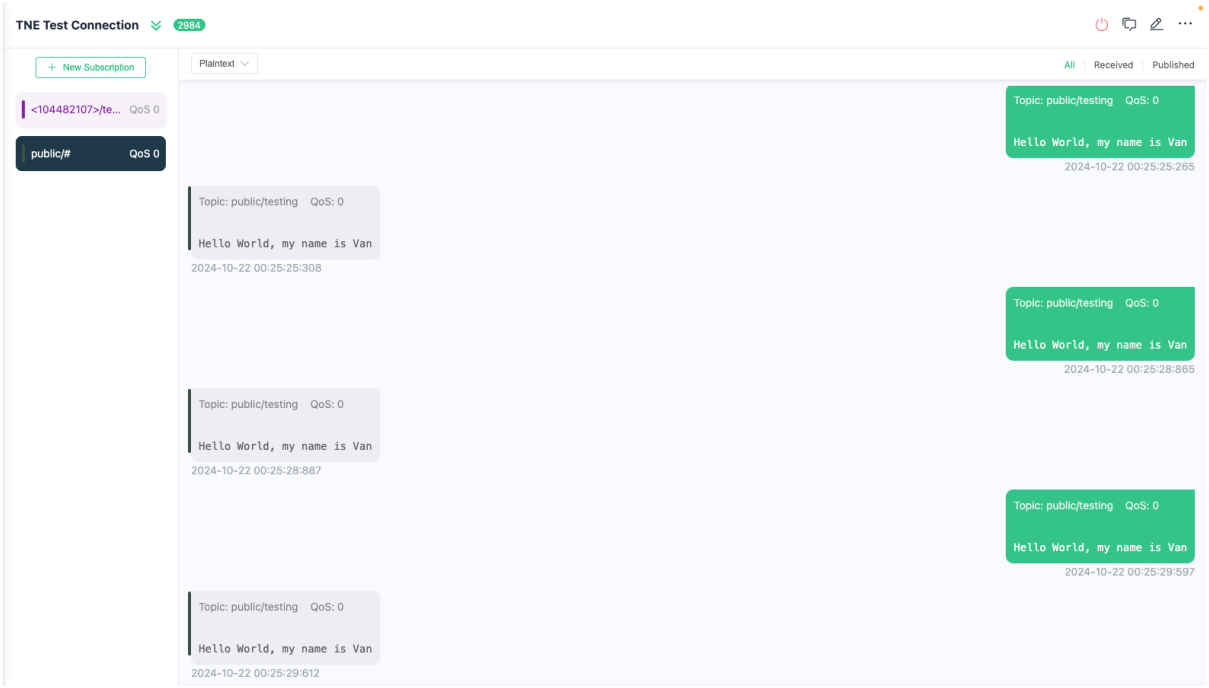
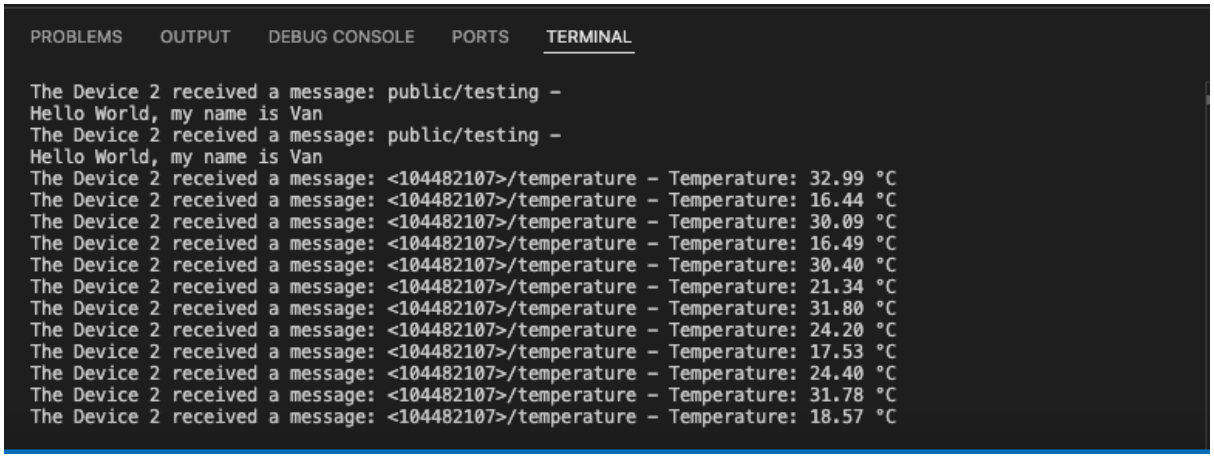


Figure 2 Device 2 receive testing message



- Source code and Instrution:

device1.py > ...

```python
import paho.mqtt.client as mqtt
import time
import random

broker = "rule28.i4t.swin.edu.au"
username = "<104482107>"
password = "<104482107>"
port = 1883
private_topic = f"{username}/temperature"

client = mqtt.Client(client_id="Device1")
client.username_pw_set(username, password)
client.connect(broker, port, 60)

client.loop_start()

try:
    while True:
        temperature = random.uniform(15, 35)
        format_temperature = f"{temperature:.2f}"
        client.publish(private_topic, f"Temperature: {format_temperature} °C")
        print(f"Published: Forecast Temperature: {format_temperature} °C to topic: {private_topic}")
        time.sleep(5)
finally:
    client.loop_stop()
    client.disconnect()
```

Figure 3 client device 1

device2.py > ...

```python
import paho.mqtt.client as mqtt

broker = "rule28.i4t.swin.edu.au"
username = "<104482107>"
password = "<104482107>"
port = 1883
public_topic = "public/#"
private_topic = f"{username}/temperature"

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("The Device 2 connected successfully")
        client.subscribe(public_topic)
        client.subscribe(private_topic)
    else:
        print(f"The Device 2 failed to connect, return code {rc}")

def on_message(client, userdata, msg):
    print(f"The Device 2 received a message: {msg.topic} - {msg.payload.decode()}")

client = mqtt.Client(client_id="Device2")
client.username_pw_set(username, password)
client.on_connect = on_connect
client.on_message = on_message

client.connect(broker, port, 60)

client.loop_forever()
```

Figure 4 client device 2

3.2 Assessment Credit

In my credit task, I have implemented a setup from the pass task where one device generates messages and subscribes to a topic, and both devices need to subscribe to the public topic. To implement this, I set device 1 to generate temperature data and post this information to a private topic (<104482107>/temperature) in the MQTT broker, as well as all publish/# the generated data to a public topic on device one. Additionally, this implementation allow device one acts as both a publisher and subscriber, and both devices are now subscribed to the public topic, then receiving the temperature information from that topic.

1. Run Device 1:
- Open a terminal and run the code for Device 1.
- Device 1 will generate temperature data and publish it to the private topic MQTT topic and subscribes to a public topic.
2. Run Device 2:
- Open another terminal and run the code for Device 2.
- Device 2 will then receive the messages being published by Device 1 and also subcribe to the public message from the public channel. (

  Noted: To run this code you have to used Swinburne network or using Swinburne VPN.

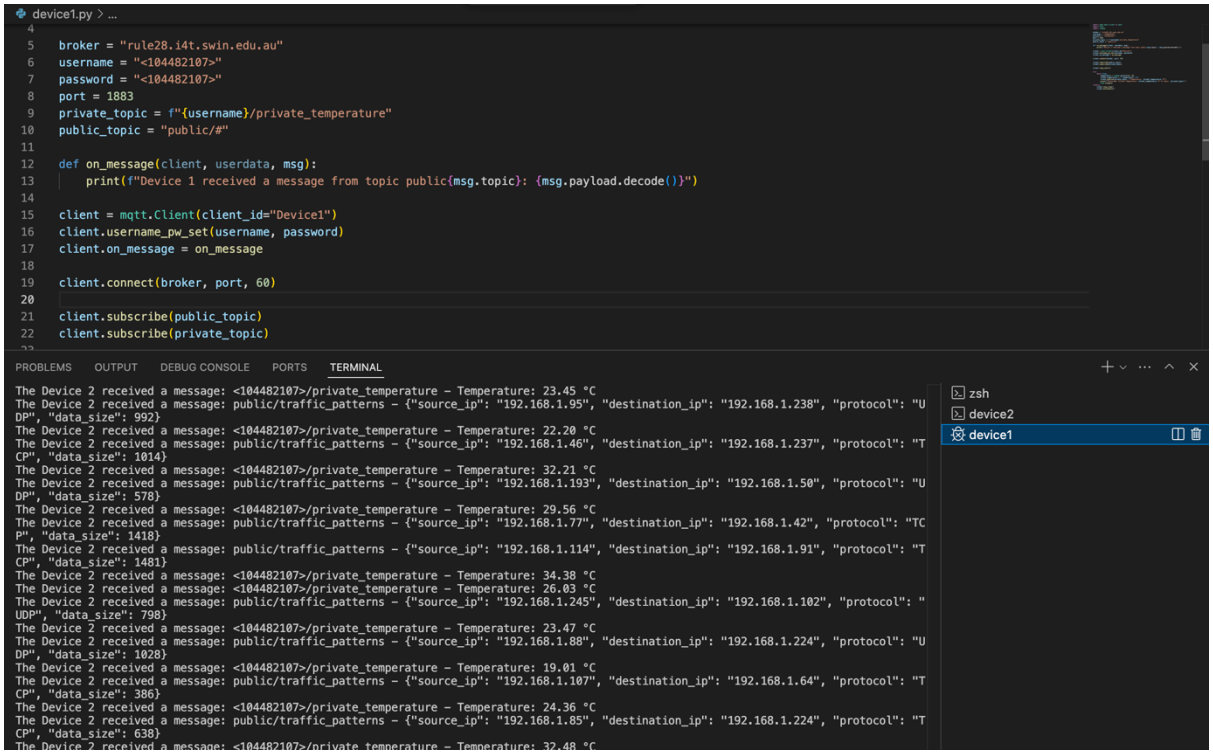
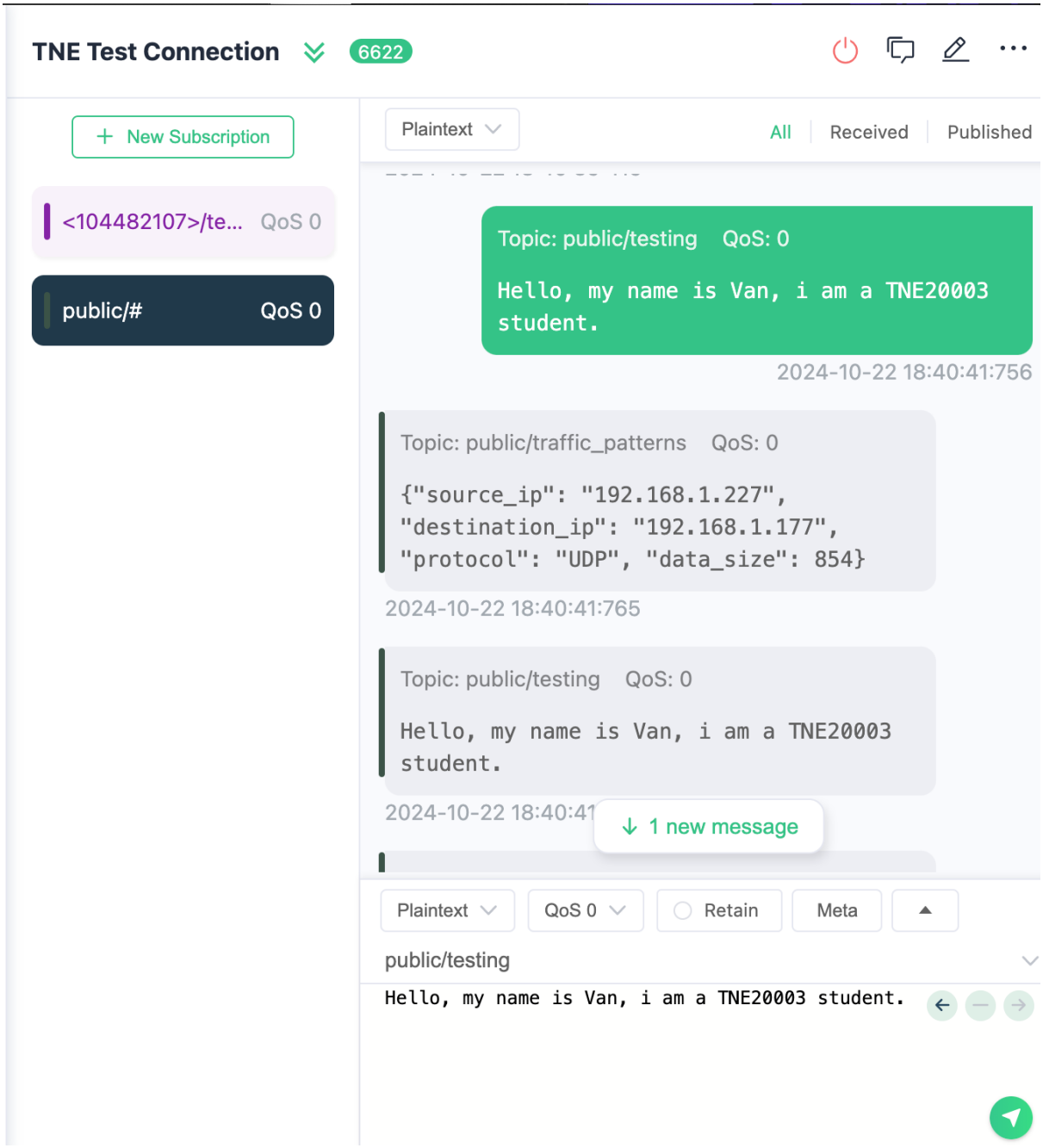
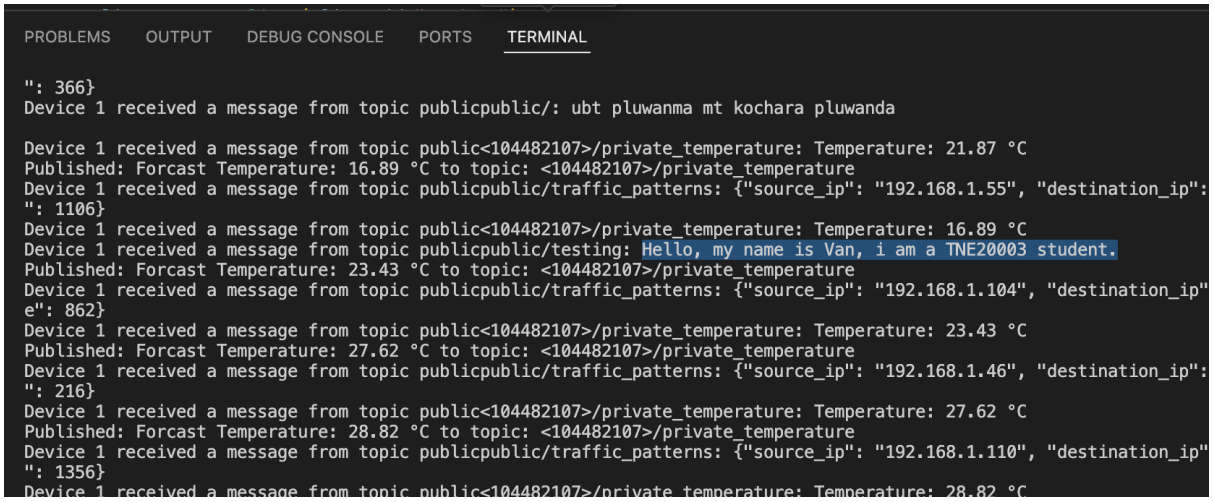


Figure 5 client device 1 acts publisher and subscriber


TNE Test Connection
6622
New Subscription
<104482107>/te... QoS 0
public/# QoS 0
Plaintext
All
Received
Published
Topic: public/testing QoS: 0
Hello, my name is Van, i am a TNE20003 student.
2024-10-22 18:40:41:756
Topic: public/traffic_patterns QoS: 0
{"source_ip": "192.168.1.227", "destination_ip": "192.168.1.177", "protocol": "UDP", "data_size": 854}
2024-10-22 18:40:41:765
Topic: public/testing QoS: 0
Hello, my name is Van, i am a TNE20003 student.
2024-10-22 18:40:41
1 new message
Plaintext
QoS 0
Retain
Meta
public/testing
Hello, my name is Van, i am a TNE20003 student.



4.Access is blocked from off-site

The MQTT broker provided by Swinburne has limited access. To connect to the Swinburne MQTT server, only the Swinburne local network is allowed. This means that you have to used Swinburne network and people outside of Swinburne cannot access it, ensuring security and preventing the exposure of sensitive information. To connect to the MQTT server from an external network, you must use the Swinburne VPN (Virtual Private Network). This VPN hides the user's IP address and encrypts the transmitted data, ensuring security and privacy.

5.Security Issues

In the current deployment of the MQTT broker, several problems need to be addressed to ensure security, reliability, and efficiency during the data exchange process when sending messages to the MQTT broker:

- Weak Authentication: The current authentication method is not secure because it uses only a username and password to establish the connection to the broker, which is easy to guess and gain unauthorized access to. This could allow attackers to gain access to the broker, leading to potential data theft. To address this issue, authentication must be strengthened by implementing more complex passwords and increasing their complexity requirements.
- Lack of Data Encrytion: The data transits as plain text over the MQTT broker and is not encrypted, which could allow attackers to intercept and view the data, potentially leading to privacy information leaks. To implement a more secure

system, TLS/SSL encryption should be used for all communication between the client and the broker.

- Not Secure Broker Config:The MQTT broker is not secure, it cloud permit any client to publish or subcribe to any topic without any restriction, this cloud lead to unauthorized access to sensitive information.

5.Conclusion

In conclusion, the main purpose of this report is to use the MQTT broker for exchanging information between devices.Moreover, i have implemented up to the credit level by allowing Device 1 to subscribe to the public channel and also generate random temperature information, which is then sent to Device 2 through the MQTT broker as a central point between the exchanging information. This implementation is to indicate the understanding of how MQTT brokers facilitate communication between IoT devices. Lastly, it demonstrates the process of setting up devices to both publish and subscribe to topics, ensuring they can exchange data efficiently.

6.Reference

Ltd, E. T. C., & Ltd, E. T. C. (n.d.). *How to use MQTT in Python (Paho)*. Www.emqx.com. https://www.emqx.com/en/blog/how-to-use-mqtt-in-python

Inc, E. T. (n.d.). *MQTTX: Your All-in-one MQTT Client Toolbox*. Mqttx.app. https://mqttx.app/